Università Ca'Foscari Venezia

Adversarial Search

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Games Vs. Planning Problems

In many problems you are are pitted against an opponent

- you cannot control your opponent's moves
- certain operators are beyond your control

You cannot search the entire space for an optimal path

- your opponent may make a move which makes any path you find obsolete

You need a strategy that leads to a winning position regardless of how your opponent plays

Games Vs. Planning Problems

Search strategies must take into account the conflicting goals of the agents

"Unpredictable" opponent: $\Rightarrow$ solution is a strategy

- Agents goals are in conflict: adversarial search (game)
- Specify a move for every possible opponent reply

Time limits: unlikely to find optimal move, must approximate

- Multiple agent Environment
- It is a search where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us.

Why Study Games in AI?

problems are formalized

real world knowledge (common sense knowledge) is not too important

rules are fixed

adversary modeling is of general importance (e.g., in economic situations, in military operations, ...)

- opponent introduces uncertainty
- programs must deal with the contingency problem

complexity of games??

- number of nodes in a search tree (e.g., 36^{40} legal positions in chess)
- specification is simple (no missing information, well-defined problem)

Types of Games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon, monopoly
imperfect information		bridge, poker, scrabble, nuclear war

We restrict our analysis to a very specific set of games:

2-player zero-sum discrete finite deterministic games of perfect information

What does it mean?

Two player: :-)

Zero-sum: In any outcome of any game, Player A's gains equal player B's losses

Discrete: All game states and decisions are discrete values

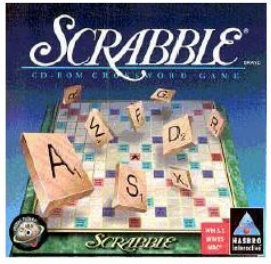
Finite: Only a finite number of states and decisions

Deterministic: No chance (no die rolls)

Perfect information: Both players can see the state, and each decision is made sequentially (no simultaneous moves)

Università
Ca'Foscari
Venezia

Types of Games

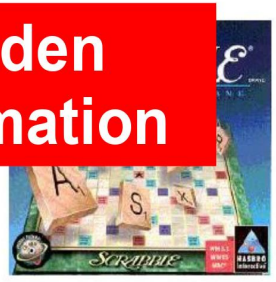


Types of Games

Not Finite

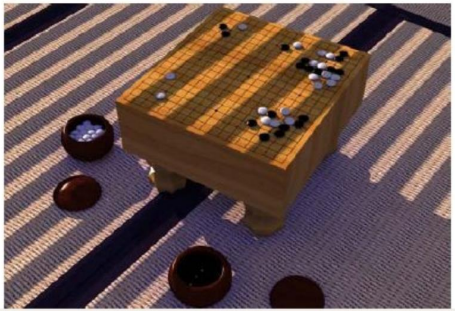
Stochastic

Hidden Information



One Player

Mutiplayer



Involves Animal Behavior

Game Tree Search

Initial state: initial board position and player

Operators: one for each legal move

Goal states: winning board positions

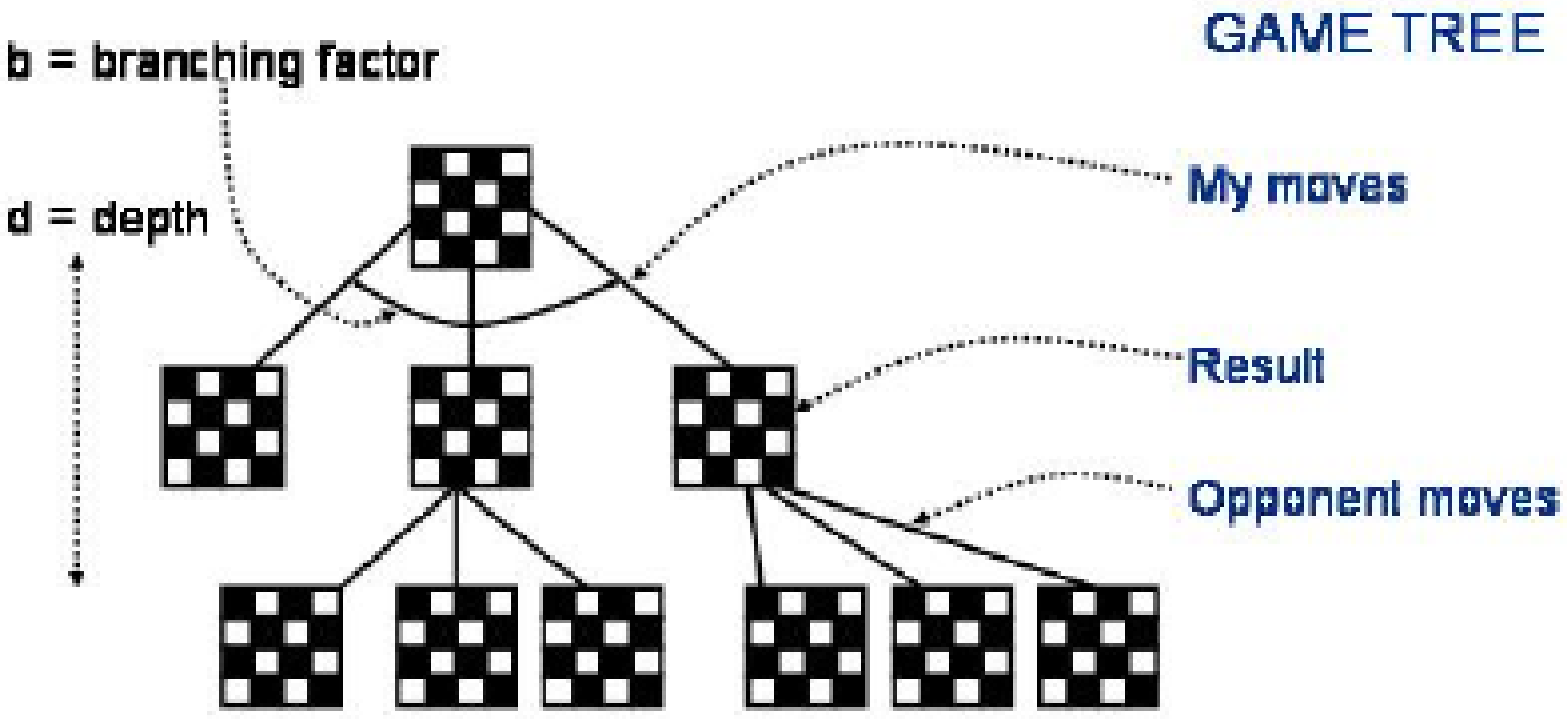
Scoring Function: assigns numeric value to states

Game tree: encodes all possible games

We are not looking for a path, only the next move to make

Our best move depends on what the other player does

Move Generation



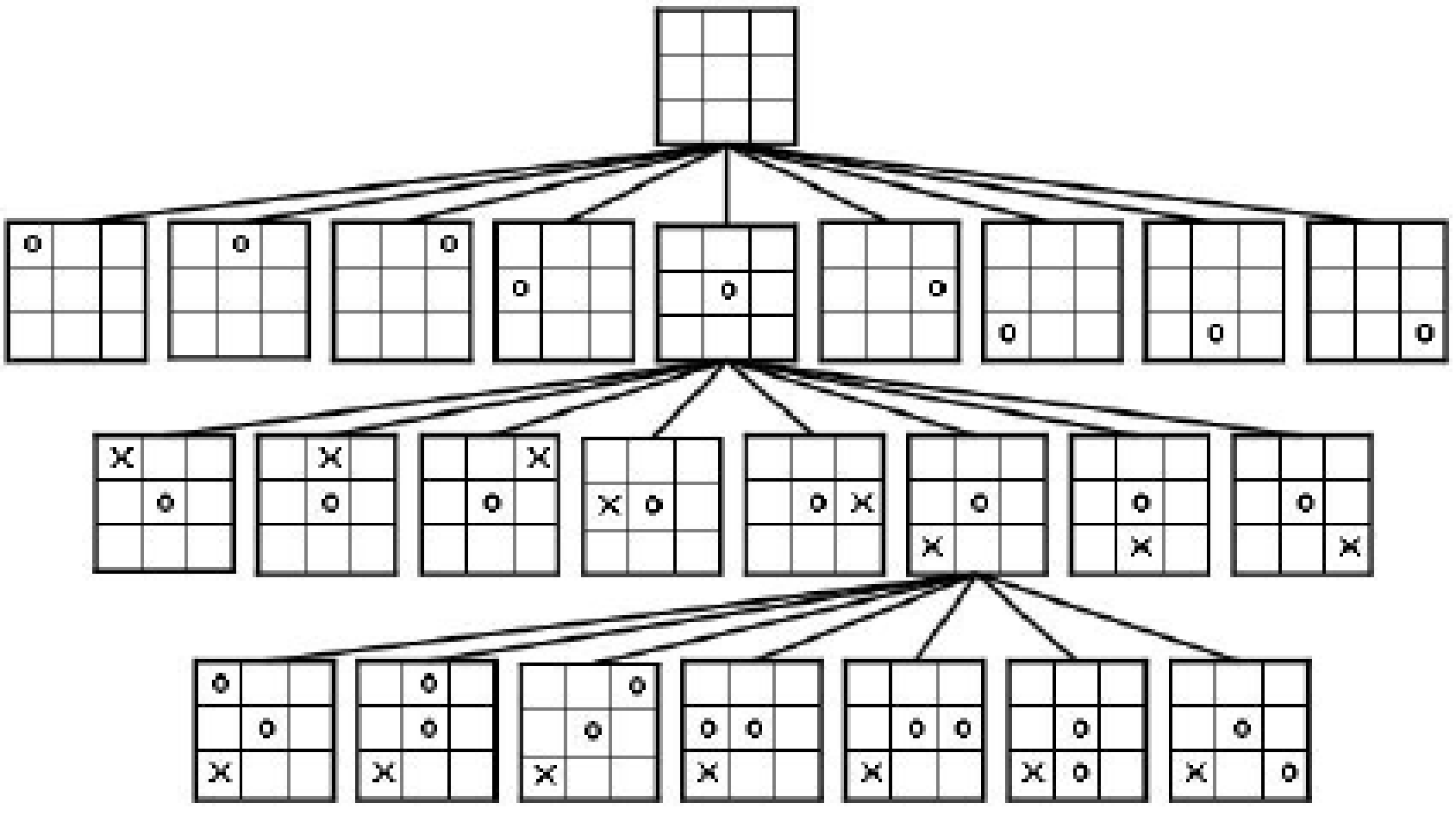
Chess

b = 36

d > 40

36^{40} is big!

Example: Tic-Tac-Toe



Minimax Criterion

Assume game tree of uniform depth (to simplify matters)

- Generate entire game tree
- Apply utility function to each terminal state
- To determine utility of nodes at any level, if Min's turn to play it will choose child with minimum utility, otherwise Max will choose child with maximum utility
- Continue backing up values from leaf to root, one level at a time

Maximizes utility under assumption that opponent will play perfectly to minimize it (assuming also opponent has same evaluation function)

Minimax Algortihm

[Minimax Algorithm

```
def minimax(state):
  value,action = max_value(state)
  return action

def max_value(state):
  if terminal(state):
    return utility(state),None
  value = -Infinity
  action=None
  for a,s in successors(state):
    v,a2 = min_value(s)
    if v>value:
      value = v
      action = a
  return value,action
```

Minimax Algortihm

Minimax Algorithm

```
def min_value(state):
  if terminal(state):
    return utility(state),None
  value = Infinity
  action=None
  for a,s in successors(state):
    v,a2 = max_value(s)
    if v<value:
      value = v
      action = a
  return value,action
```

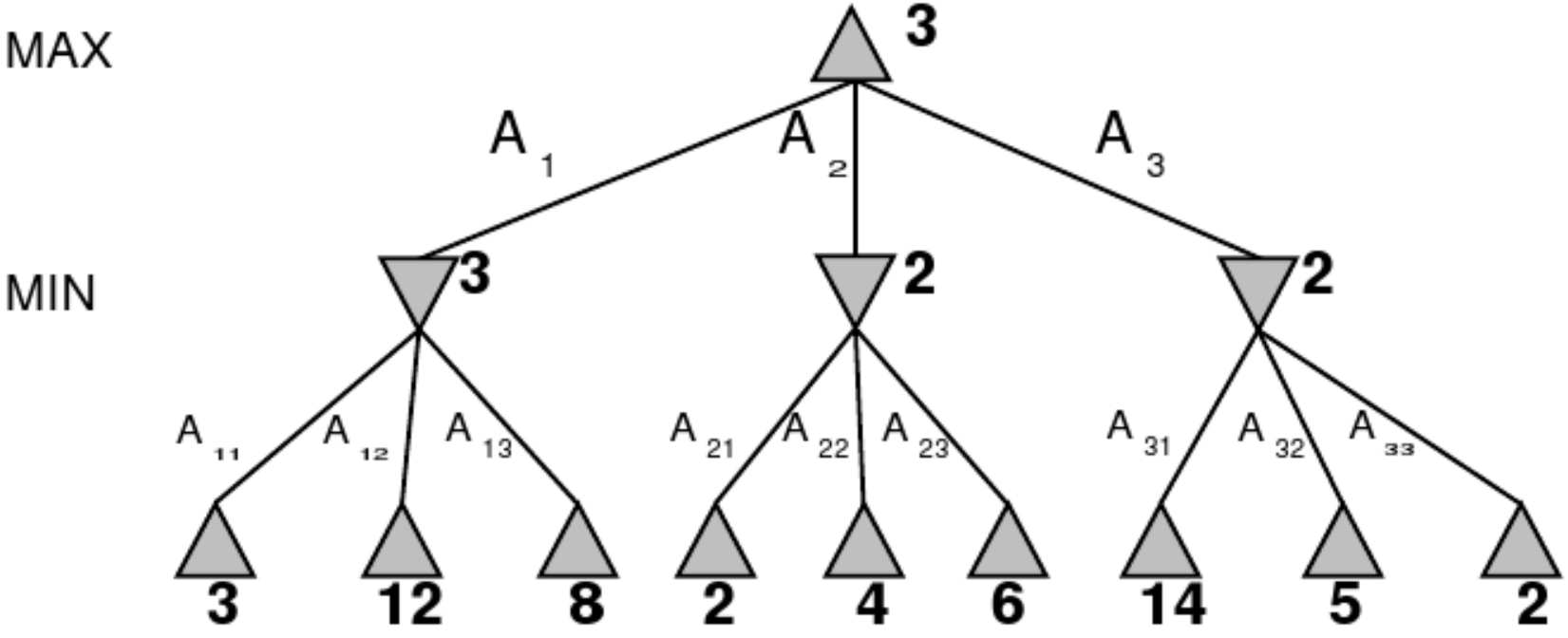
Minimax

Perfect play for deterministic, perfect-information games

Idea: choose move to position with highest minimax value

- best achievable payoff against best play

E.g., 2-ply game:



Properties of Minimax

Complete: Yes, if tree is finite (chess has specific rules for this)

Optimal: Yes, against an optimal opponent (Otherwise??)

Time complexity: $O(b^d)$

Space complexity: $O(db)$ (depth-first exploration)

For chess, $b \approx 35$, $d \approx 40$ for "reasonable" games

- exact solution completely infeasible

Resource Limits

Standard approach:

- **cutoff test** (*e.g.*, depth limit)
- **evaluation function** estimated desirability of position and explore only (hopeful) nodes with certain values

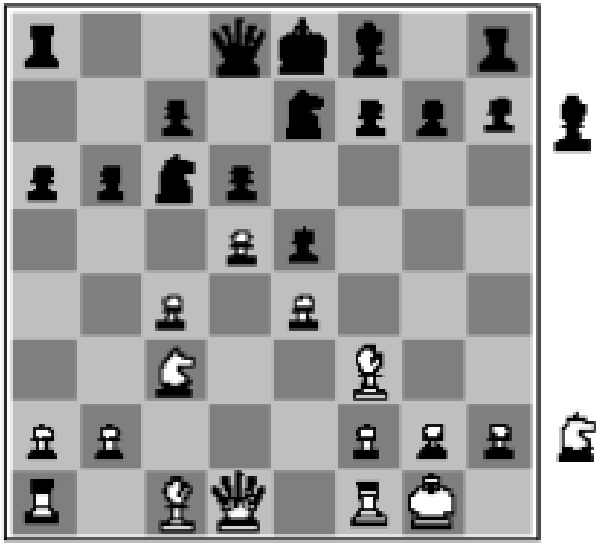
`MINIMAX-CUTOFF` is identical to `MINIMAX` except

- `TERMINAL-TEST` is replaced by `CUTOFF-TEST`
- `UTILITY` is replaced by `EVAL`

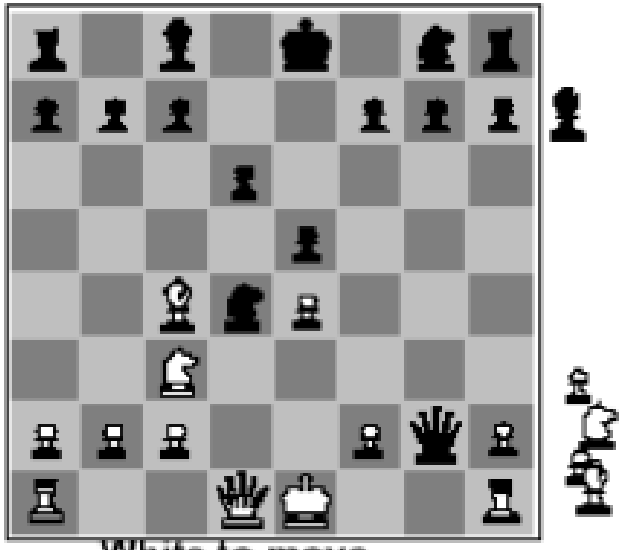
Search depth in chess:

- 4-ply $\approx$ human novice
- 8-ply $\approx$ typical PC, human master
- 12-ply $\approx$ Deep Blue, Kasparov

Evaluation functions



Black to move
White slightly better



White to move
Black winning

For chess, typically linear weighted sum of features

$$\text{eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

e.g., $f_1(s) =$ (number of white queens) – (number of black queens)

Other features which could be taken into account: number of threats, good structure of pawns, measure of safety of the king.

Università
Ca'Foscari
Venezia

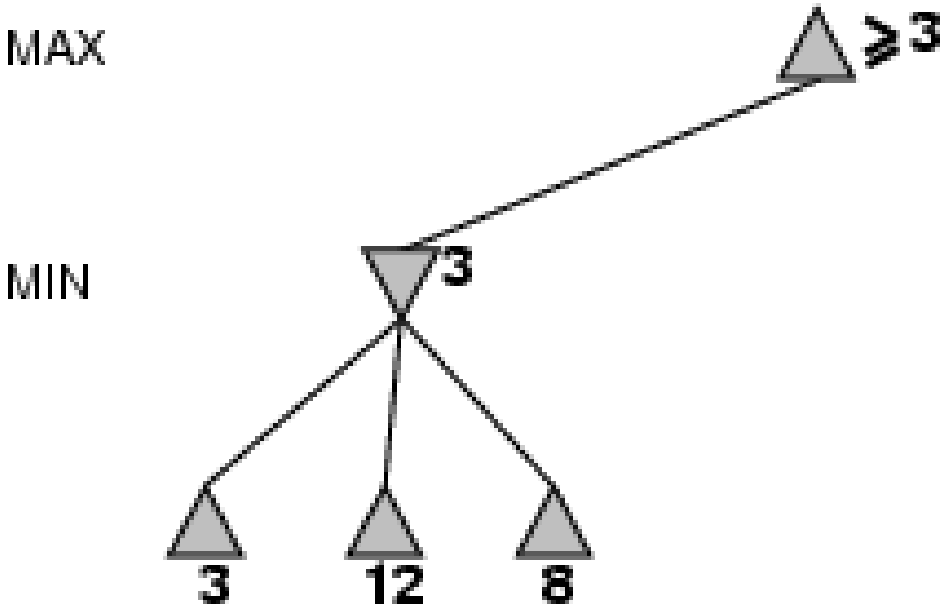
α-β Pruning

The problem with minimax algorithm search is that the number of game states it has to examine is exponential in the number of moves:

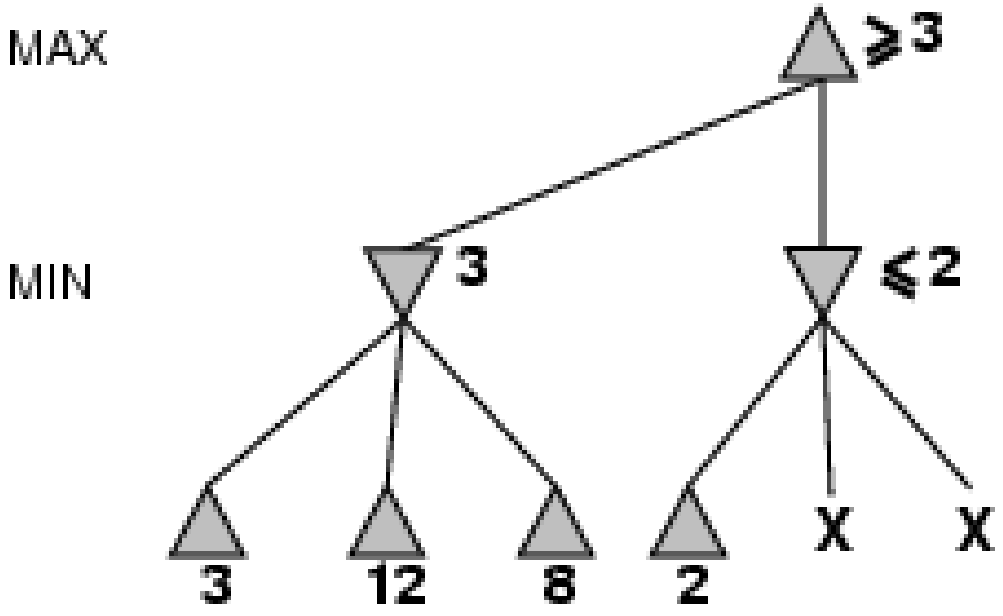
α-β proposes to compute the correct minimax algorithm decision without looking at every node in the game tree.

PRUNING!

Example of α-β Pruning

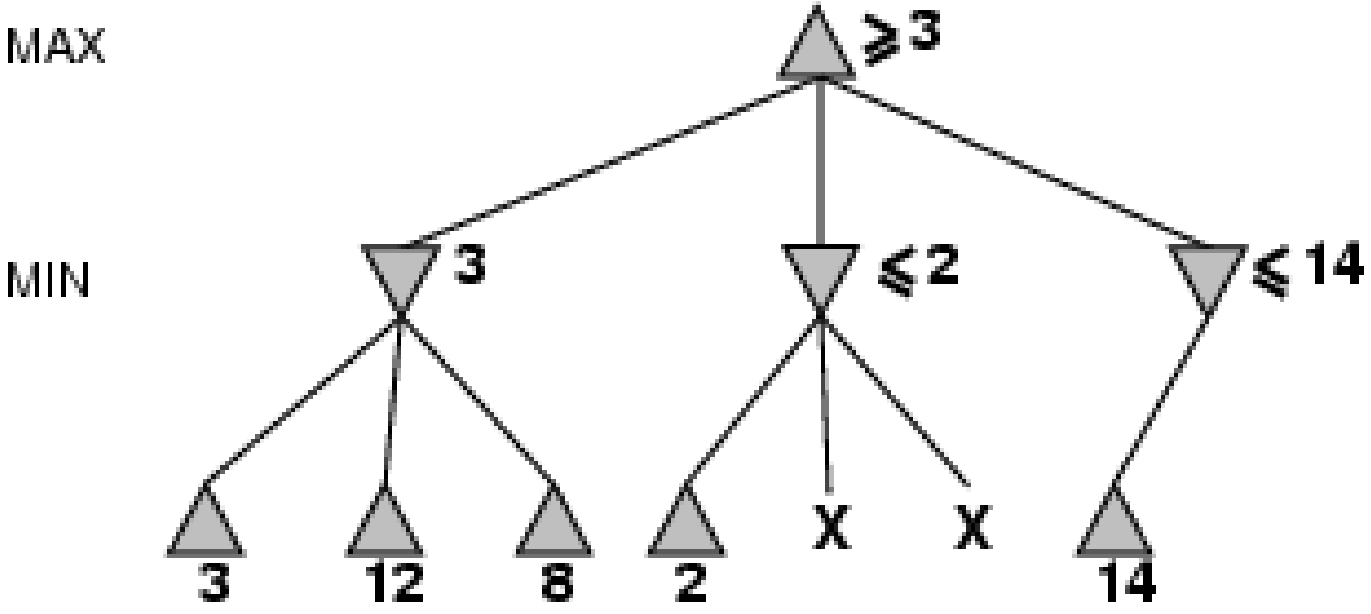


Example of α-β Pruning



Università
Ca'Foscari
Venezia

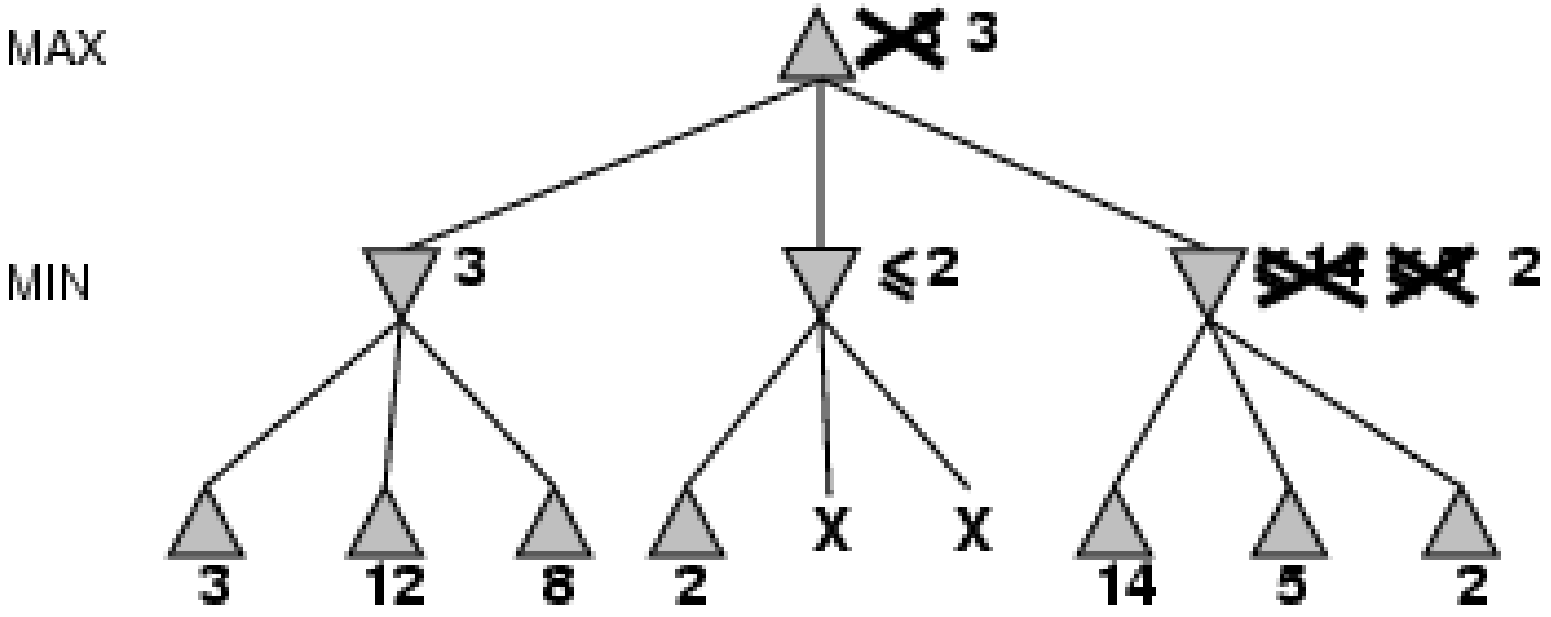
Example of α-β Pruning



We see: possibility to prune depends on the order of processing the successors!

Università
Ca'Foscari
Venezia

Example of α-β Pruning



We see: possibility to prune depends on the order of processing the successors!

Properties of α-β Pruning

Pruning does not affect final result

Good move ordering improves effectiveness of pruning

With "perfect ordering," time complexity = $O(b^{m/2})$

- doubles possible depth of search doable in the same time

A simple example of the value of reasoning about which computations are relevant (meta-reasoning)

α-β Algortihm

α-β Algorithm

```
class AlphaBeta:
  def__init__(self):
        # set initial alpha=-Infinity and
          beta=Infinity
    self.alpha = -Infinity
    self.beta = Infinity

def alpha-beta(state):
  value,action = max_value(state, AlphaBeta())
  return action
```

α-β Algortihm

α-β Algorithm

```
def max_value(state, alpha_beta):
  if terminal(state):
    return utility(state),None
  value = -Infinity
  action=None
  for a,s in successors(state):
    v,a2 = min_value(s, alpha_beta)
    if v>value:
      value = v
      action = a
    if value >= alpha_beta.beta:
      return value,action
    alpha_beta.alpha=max(AlphaBeta.alpha, value)
  return value,action
```

α-β Algortihm

α-β Algorithm

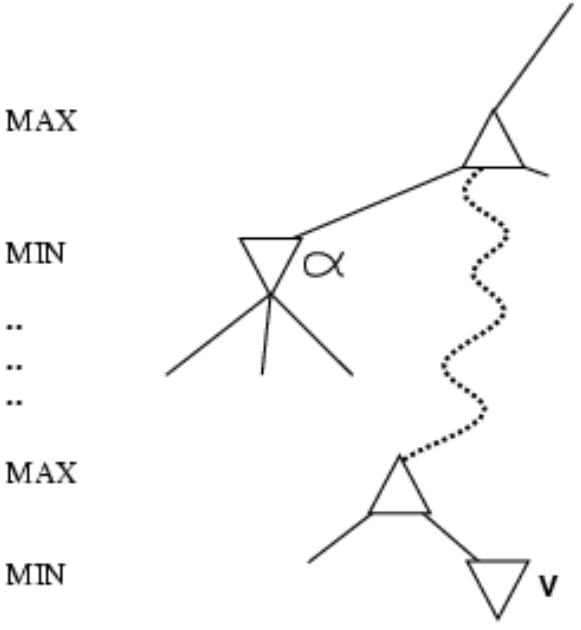
```
def min_value(state, alpha_beta):
  if terminal(state):
    return utility(state),None
  value = Infinity
  action=None
  for a,s in successors(state):
    v,a2 = max_value(s, alpha_beta)
    if v<value:
      value = v
      action = a
    if value <= alpha_beta.alpha:
      return value,action
    alpha_beta.beta = min(AlphaBeta.beta, value)
  return value,action
```

Why is it called α-β?

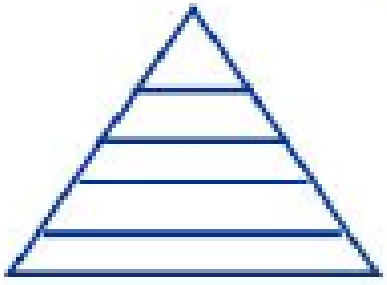
α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for max

If `value` is worse than α, max will return immediately (prune that branch)

Define β similarly for min



Practical Matters



Variable branching

- **Iterative deepening**
  - order best move from last search first
  - use previous backed up value to initialize $[\alpha, \beta]$
  - keep track of repeated positions (transposition tables)

Horizon effect

- quiescence
  - Pushing the inevitable over search horizon

Parallelization

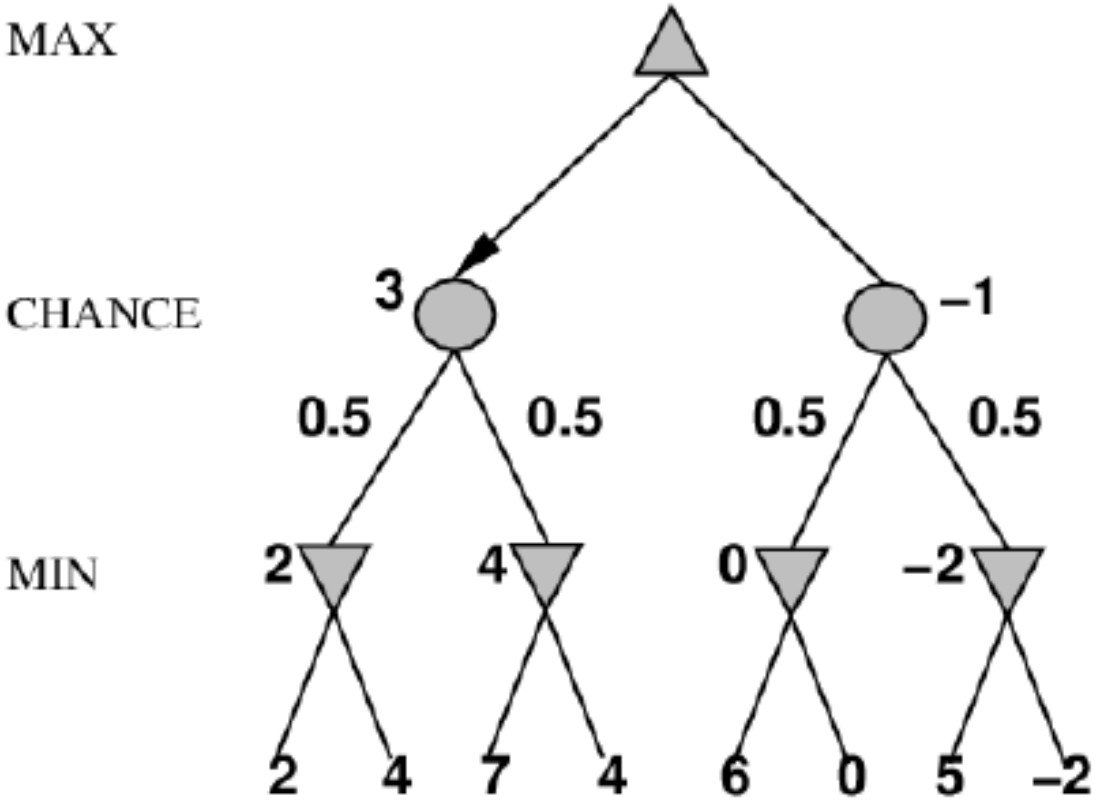
Università
Ca'Foscari
Venezia

Non-Deterministic Games

E.g., backgammon: dice rolls determine legal moves

We can do Minimax with a extra "chance" layer

Simplified example with coin-flipping instead of dice-rolling



Expect-Minimax Algorithm

Expect-Minimax gives perfect play for non-deterministic games

Like Minimax, except add chance nodes

- For max node return highest `expect_minimax` of `successors`
- For min node return lowest `expect_minimax` of `successors`
- For chance node return average of `expect_minimax` of `successors`

Here exact values of evaluation function do matter ("probabilities", "expected gain", not just order)

α-β pruning possible by taking weighted averages according to probabilities

Games of Imperfect Information

E.g., card games (bridge, hearts, some forms of poker)

- Opponent's initial cards are unknown
- Not quite like non-deterministic games

We can calculate probabilities for each possible deal
(Seems just like one big dice roll at the beginning)

- Compute the minimax value of each action in each deal
- Then choose action with highest expected value over all deals
- Special case: an action optimal for all deals, is optimal

Take weighted average over all possible situations to make decision at the top of the game tree

Requires a lot of computation. . .